

Core Service Integrations

Three integrations carry the system: Odoo over XML-RPC, Mistral for OCR, and the matching engine that reconciles them. Each has one non-obvious constraint that shapes its design.

Service	The constraint that shapes it
<code>odoo_service.py</code>	<code>xmlrpc.client</code> is fully blocking — it must never touch the event loop
<code>ocr_service.py</code>	<code>mistralai</code> v2 moved its imports; annotation is capped at 8 pages
<code>matching_engine.py</code>	Must be pure (no I/O) or it cannot be tested or tuned
<code>kb_service.py</code>	Learning must be idempotent under concurrency

Text normalization — `app/utils/text.py`

Everything downstream depends on this being right. `normalize_company_name` generates the knowledge base's primary key, so its behaviour is effectively schema.

```
from __future__ import annotations

import re
import unicodedata

# Ordered longest-first so "PRIVATE LIMITED" is stripped before "LIMITED" can
# match inside it and leave a stray "PRIVATE" behind.
_LEGAL_SUFFIXES = [
    "PRIVATE LIMITED", "PVT LTD", "PVT. LTD.", "PUBLIC LIMITED COMPANY",
    "SOCIETE ANONYME", "LIMITED LIABILITY COMPANY", "AND SONS", "AND CO",
    "INCORPORATED", "CORPORATION", "COMPANY", "LIMITED", "HOLDINGS", "GROUP",
    "GMBH & CO KG", "GMBH", "AKTIENGESELLSCHAFT", "S DE RL DE CV", "SA DE CV",
    "LLC", "L L C", "LTD", "INC", "CORP", "PLC", "LLP", "PTE", "PTY", "BV",
    "NV", "AG", "SA", "SL", "SRL", "SPA", "OY", "AB", "AS", "KG", "CO",
]

_SUFFIX_RE = re.compile(
    r"\b(" + "|".join(re.escape(s) for s in _LEGAL_SUFFIXES) + r")\b\.", re.IGNORECASE
)
_NOISE_RE = re.compile(r"^[A-Z0-9 ]+")
_WS_RE = re.compile(r"\s+")
_PO_REF_RE = re.compile(
    r"\b(?:P\.\?\.?|PURCHASE\s*ORDER)[\s#:\.-]*([A-Z0-9][A-Z0-9\-/]{2,})\b", re.I
)

def strip_accents(value: str) -> str:
    return "".join(
        c for c in unicodedata.normalize("NFKD", value) if not unicodedata.combining(c)
    )

def normalize_company_name(value: str | None) -> str:
    """'Acme Corp. (Pvt) Ltd.' -> 'ACME'.


This is the knowledge base's primary key generator. Changing it invalidates
    every stored normalized_key - if you ever do, ship a data migration that
    recomputes them all.


    """
    if not value:
        return ""
    out = strip_accents(value).upper()
    out = out.replace("&", " AND ")
    out = _NOISE_RE.sub(" ", out)
    out = _WS_RE.sub(" ", out).strip()
```

```

# Loop: 'FOO LTD CO' needs two passes, because stripping 'LTD' is what
# exposes 'CO' at a word boundary.
prev = None
while prev != out:
    prev = out
    out = _SUFFIX_RE.sub(" ", out)
    out = _WS_RE.sub(" ", out).strip()

# Guard against a name that is ENTIRELY legal suffixes, e.g. a vendor
# literally called "Holdings Group Ltd" - returning "" would collide with
# every other empty key in the KB's unique constraint.
return out or _WS_RE.sub(
    " ", _NOISE_RE.sub(" ", strip_accents(value).upper())
).strip()

def normalize_description(value: str | None) -> str:
    if not value:
        return ""
    out = strip_accents(value).upper()
    out = re.sub(r"^[A-Z0-9 ]+", " ", out)
    return _WS_RE.sub(" ", out).strip()

def normalize_reference(value: str | None) -> str:
    """'PO-000 42/A' -> 'P000042A', so P000042 and po/00042 compare equal."""
    if not value:
        return ""
    return re.sub(r"^[A-Z0-9]", "", strip_accents(value).upper())

def extract_po_references(text: str | None) -> list[str]:
    """Pull every PO-looking token out of the raw OCR markdown.

    Vendors print the PO number in wildly inconsistent places - header, footer,
    inside a line item description. Scanning the full text catches the ones the
    structured extraction missed.
    """
    if not text:
        return []
    return list({normalize_reference(m.group(1)) for m in _PO_REF_RE.finditer(text)})

```

Worth unit-testing explicitly, because these are the cases that break in production:

Input	Expected
Acmé Corp. (Pvt) Ltd.	ACME
ACME INTERNATIONAL GMBH & CO KG	ACME INTERNATIONAL
Smith & Sons Limited	SMITH AND
Holdings Group Ltd	falls back to HOLDINGS GROUP LTD, not ""
PO-000 42/A (as reference)	P000042A

Odoo — `app/services/odoo_service.py`

The single most important detail in this file: `xmlrpc.client` is fully blocking. One 30-second Odoo call made directly on the event loop freezes every other request in the process. The design is therefore a sync client class wrapped by an async facade that offloads every socket touch to a worker thread.

```

from __future__ import annotations

import datetime as dt
import socket
import ssl

```

```

import threading
import xmlrpc.client
from decimal import Decimal
from functools import partial
from typing import Any

import anyio
import structlog
from tenacity import (
    RetryError, retry, retry_if_exception_type, stop_after_attempt,
    wait_exponential_jitter,
)

from app.core.config import settings
from app.core.errors import OdooAuthError, OdooError, OdooUnavailableError
from app.schemas.odoo import OdooPartner, OdooPOLine, OdooPurchaseOrder

logger = structlog.get_logger(__name__)

# Retry transport hiccups only – never a Fault, which means Odoo understood the
# request and rejected it. Retrying a bad domain just fails three times slower.
_TRANSIENT = (
    socket.timeout, socket.gaierror, ConnectionError, OSError,
    xmlrpc.client.ProtocolError, ssl.SSLError,
)

PO_FIELDS = [
    "id", "name", "partner_id", "partner_ref", "date_order", "date_planned",
    "state", "invoice_status", "currency_id", "amount_untaxed", "amount_tax",
    "amount_total", "order_line", "company_id",
]
PO_LINE_FIELDS = [
    "id", "name", "product_id", "product_qty", "qty_received", "qty_invoiced",
    "price_unit", "price_subtotal", "price_total", "product_uom", "order_id",
]
OPEN_PO_STATES = ["draft", "sent", "to approve", "purchase"]

def _m2o_id(v: Any) -> int | None:
    """Odoo many2one fields come back as [id, "Display Name"], or False when unset."""
    return int(v[0]) if isinstance(v, (list, tuple)) and v else None

def _m2o_name(v: Any) -> str | None:
    return str(v[1]) if isinstance(v, (list, tuple)) and len(v) > 1 else None

def _dec(v: Any) -> Decimal:
    # Odoo sends False rather than null for empty numerics.
    if v in (False, None, ""):
        return Decimal("0")
    return Decimal(str(v))

def _dt(v: Any) -> dt.datetime | None:
    if not v or v is False:
        return None
    # Odoo serializes naive UTC as 'YYYY-MM-DD HH:MM:SS'.
    for fmt in ("%Y-%m-%d %H:%M:%S", "%Y-%m-%d"):
        try:
            return dt.datetime.strptime(str(v), fmt).replace(tzinfo=dt.UTC)
        except ValueError:
            continue
    return None

class OdooCredentials:
    __slots__ = ("url", "db", "username", "api_key")

    def __init__(self, url: str, db: str, username: str, api_key: str) -> None:
        self.url = url.rstrip("/")
        self.db = db
        self.username = username
        self.api_key = api_key

@property

```

```
def cache_key(self) -> tuple[str, str, str]:
    return (self.url, self.db, self.username)
```

Authentication and the RPC core

```
class _BlockingOdooClient:
    """Pure-sync XML-RPC client.

    NEVER call any method of this class directly from async code - always go
    through OdooService's threadpool wrapper below.
    """

    def __init__(self, creds: OdooCredentials, timeout: int) -> None:
        self._creds = creds
        self._timeout = timeout
        self._uid: int | None = None
        self._lock = threading.Lock()
        # allow_none=True is mandatory: Odoo returns null in plenty of places and
        # xmlrpc refuses to marshal None without it.
        self._common = xmlrpc.client.ServerProxy(
            f"{creds.url}/xmlrpc/2/common", allow_none=True
        )
        self._models = xmlrpc.client.ServerProxy(
            f"{creds.url}/xmlrpc/2/object", allow_none=True
        )

    # ----- auth
    def authenticate(self, *, force: bool = False) -> int:
        with self._lock:
            if self._uid is not None and not force:
                return self._uid

            # ServerProxy has no timeout parameter, so the socket default is the
            # only lever. Save and restore it so we don't leak a global setting.
            prev = socket.setdefaulttimeout()
            socket.setdefaulttimeout(self._timeout)
            try:
                uid = self._common.authenticate(
                    self._creds.db, self._creds.username, self._creds.api_key, {}
                )
            except _TRANSIENT as exc:
                raise OdooUnavailableError(
                    f"Cannot reach Odoo at {self._creds.url}: {exc}"
                ) from exc
            except xmlrpc.client.Fault as exc:
                raise OdooAuthError(
                    f"Odoo rejected authentication: {exc.faultString}"
                ) from exc
            finally:
                socket.setdefaulttimeout(prev)

            # Odoo returns False (not an error) for bad credentials.
            if not uid:
                raise OdooAuthError(
                    "Odoo authentication returned no uid - check the database "
                    "name, login and API key."
                )
            self._uid = int(uid)
            return self._uid

    def version(self) -> dict[str, Any]:
        return self._common.version()

    # ----- core rpc
    @retry(
        retry=retry_if_exception_type((*_TRANSIENT, OdooUnavailableError)),
        stop=stop_after_attempt(settings.ODOO_MAX_RETRIES),
        wait=wait_exponential_jitter(initial=0.5, max=6.0),
        reraise=True,
    )
    def execute_kw(
        self,
        model: str,
        method: str,
```

```

args: list[Any] | None = None,
kwargs: dict[str, Any] | None = None,
) -> Any:
    uid = self.authenticate()
    prev = socket.getdefaulttimeout()
    socket.setdefaulttimeout(self._timeout)
    try:
        return self._models.execute_kw(
            self._creds.db, uid, self._creds.api_key,
            model, method, args or [], kwargs or {},
        )
    except xmlrpc.client.Fault as exc:
        fault = exc.faultString or ""
        # Odoo raises AccessDenied as a Fault when the API key was revoked.
        # Clearing the cached uid means the next call re-authenticates rather
        # than looping on a dead session.
        if "AccessDenied" in fault or "Access Denied" in fault:
            self._uid = None
            raise OdooAuthError(
                "Odoo access denied – the API key may have been revoked."
            ) from exc
        if "AccessError" in fault:
            raise OdooError(
                f"Odoo permission error on {model}.{method}. The integration "
                f"user needs Purchase and Accounting access rights."
            ) from exc
        # Odoo tracebacks are enormous; the last line carries the message.
        raise OdooError(
            f"Odoo {model}.{method} failed: {fault.strip().splitlines()[-1]}"
        ) from exc
    except _TRANSIENT as exc:
        raise OdooUnavailableError(
            f"Odoo transport error on {model}.{method}: {exc}"
        ) from exc
    finally:
        socket.setdefaulttimeout(prev)

```

Fetching Purchase Orders with their lines

The performance-critical part. Odoo returns `order_line` as a list of ids, and the naive implementation reads lines per PO — 200 POs becomes 200 blocking HTTP round trips.

```

def search_read(
    self, model: str, domain: list[Any], fields: list[str],
    limit: int = 0, offset: int = 0, order: str | None = None,
) -> list[dict[str, Any]]:
    kwargs: dict[str, Any] = {"fields": fields, "offset": offset}
    if limit:
        kwargs["limit"] = limit
    if order:
        kwargs["order"] = order
    return self.execute_kw(model, "search_read", [domain], kwargs)

def fetch_open_purchase_orders(
    self,
    *,
    partner_ids: list[int] | None = None,
    since: dt.datetime | None = None,
    limit: int = 200,
    include_lines: bool = True,
) -> list[OdooPurchaseOrder]:
    domain: list[Any] = [
        ("state", "in", OPEN_PO_STATES),
        # Fully-billed POs cannot match a new invoice. Excluding them here
        # removes most false positives before scoring even starts.
        ("invoice_status", "!=", "invoiced"),
    ]
    if partner_ids:
        domain.append(("partner_id", "in", partner_ids))
    if since:
        domain.append(("date_order", ">=", since.strftime("%Y-%m-%d %H:%M:%S")))

    raw_pos = self.search_read(

```

```

        "purchase.order", domain, PO_FIELDS, limit=limit, order="date_order desc"
    )
    if not raw_pos:
        return []

    lines_by_order: dict[int, list[OdooPOLine]] = {}
    if include_lines:
        line_ids: list[int] = []
        for po in raw_pos:
            line_ids.extend(po.get("order_line") or [])

        if line_ids:
            # ONE batched read for every line of every PO. Never loop per PO:
            # 200 POs x 1 round trip each = 200 sequential blocking calls,
            # which turns a 2-second fetch into a 3-minute one.
            raw_lines = self.execute_kw(
                "purchase.order.line", "read", [line_ids],
                {"fields": PO_LINE_FIELDS},
            )
            for rl in raw_lines:
                order_id = _m2o_id(rl.get("order_id"))
                if order_id is None:
                    continue
                lines_by_order.setdefault(order_id, []).append(
                    OdooPOLine(
                        id=int(rl["id"]),
                        name=str(rl.get("name") or ""),
                        product_id=_m2o_id(rl.get("product_id")),
                        product_name=_m2o_name(rl.get("product_id")),
                        product_qty=float(rl.get("product_qty") or 0.0),
                        qty_received=float(rl.get("qty_received") or 0.0),
                        qty_invoiced=float(rl.get("qty_invoiced") or 0.0),
                        price_unit=_dec(rl.get("price_unit")),
                        price_subtotal=_dec(rl.get("price_subtotal")),
                        price_total=_dec(rl.get("price_total")),
                        product_uom_name=_m2o_name(rl.get("product_uom")),
                    )
                )

    return [
        OdooPurchaseOrder(
            id=int(po["id"]),
            name=str(po.get("name") or ""),
            partner_id=_m2o_id(po.get("partner_id")) or 0,
            partner_name=_m2o_name(po.get("partner_id")) or "",
            partner_ref=(po.get("partner_ref") or None),
            date_order=_dt(po.get("date_order")),
            date_planned=_dt(po.get("date_planned")),
            state=str(po.get("state") or ""),
            invoice_status=(po.get("invoice_status") or None),
            currency_id=_m2o_id(po.get("currency_id")),
            currency_name=_m2o_name(po.get("currency_id")),
            amount_untaxed=_dec(po.get("amount_untaxed")),
            amount_tax=_dec(po.get("amount_tax")),
            amount_total=_dec(po.get("amount_total")),
            order_line=lines_by_order.get(int(po["id"]), []),
        )
        for po in raw_pos
    ]

def search_partners(
    self, query: str | None = None, limit: int = 50
) -> list[OdooPartner]:
    domain: list[Any] = [("supplier_rank", ">", 0), ("active", "=", True)]
    if query:
        # Odoo domains are prefix-notation: the leading "|" makes the next
        # TWO leaves an OR.
        domain.insert(0, "|")
        domain.append(("name", "ilike", query))
        domain.append(("vat", "ilike", query))
    rows = self.search_read(
        "res.partner", domain,
        ["id", "name", "vat", "email", "supplier_rank"],
        limit=limit, order="supplier_rank desc, name asc",
    )
    return [
        OdooPartner(

```

```

        id=int(r["id"]),
        name=str(r.get("name") or ""),
        vat=(r.get("vat") or None),
        email=(r.get("email") or None),
        supplier_rank=int(r.get("supplier_rank") or 0),
    )
    for r in rows
]

```

Pushing the vendor bill back

```

def create_vendor_bill_from_po(
    self,
    po_id: int,
    *,
    vendor_ref: str | None,
    invoice_date: dt.date | None,
    post_bill: bool = False,
) -> dict[str, Any]:
    """Let Odoo build the bill, then stamp our OCR'd reference on it.

    Calling action_create_invoice means Odoo computes taxes, analytic
    accounts, currency, fiscal position and the purchase_line_id back-links
    with its own logic. Constructing account.move by hand is possible, but
    you then inherit responsibility for every one of those computations -
    don't, unless a customer's flow genuinely requires it.
    """
    before = self.execute_kw(
        "purchase.order", "read", [[po_id]],
        {"fields": ["invoice_ids", "name", "state"]},
    )
    if not before:
        raise OdooError(f"Purchase order {po_id} not found in Odoo.")
    po_name = before[0].get("name")
    existing_ids = set(before[0].get("invoice_ids") or [])

    if before[0].get("state") in ("draft", "sent"):
        # Odoo refuses to bill an unconfirmed PO.
        self.execute_kw("purchase.order", "button_confirm", [[po_id]])

    # This returns an ir.actions dict, NOT the new bill's id - hence the
    # before/after diff of invoice_ids below.
    self.execute_kw(
        "purchase.order", "action_create_invoice", [[po_id]],
        {"context": {"active_model": "purchase.order", "active_ids": [po_id]}},
    )

    after = self.execute_kw(
        "purchase.order", "read", [[po_id]], {"fields": ["invoice_ids"]}
    )
    new_ids = [
        i for i in (after[0].get("invoice_ids") or []) if i not in existing_ids
    ]
    if not new_ids:
        raise OdooError(
            f"Odoo created no vendor bill for {po_name}. The PO may be fully "
            f"invoiced or have nothing left to bill."
        )
    bill_id = int(new_ids[0])

    values: dict[str, Any] = {}
    if vendor_ref:
        values["ref"] = vendor_ref  # the vendor's invoice number
    if invoice_date:
        values["invoice_date"] = invoice_date.isoformat()
    if values:
        self.execute_kw("account.move", "write", [[bill_id], values])

    # Left in draft by default. Posting makes a live journal entry that has
    # to be reversed if the match was wrong.
    if post_bill:
        self.execute_kw("account.move", "action_post", [[bill_id]])

    bill = self.execute_kw(

```

```

        "account.move", "read", [[bill_id]],
        {"fields": ["id", "name", "state", "ref", "amount_total",
                    "invoice_date", "partner_id"]},
    )[0]
    return {
        "bill_id": bill_id,
        "bill_name": str(bill.get("name") or "/"),
        "state": bill.get("state"),
        "ref": bill.get("ref") or None,
        "amount_total": float(bill.get("amount_total") or 0.0),
        "po_name": po_name,
    }

def flag_purchase_order(self, po_id: int, note: str) -> bool:
    """Non-destructive alternative to billing: post a chatter note. Useful
    when finance bills manually and only wants the match recorded."""
    self.execute_kw(
        "purchase.order", "message_post", [[po_id]],
        {"body": note, "message_type": "comment"},
    )
    return True

```

The async facade

```

_client_cache: dict[tuple[str, str, str], _BlockingOdooClient] = {}
_cache_lock = threading.Lock()

def _get_client(creds: OdooCredentials) -> _BlockingOdooClient:
    # Cached per (url, db, username) so the authenticated uid survives across
    # requests instead of re-authenticating on every call.
    with _cache_lock:
        client = _client_cache.get(creds.cache_key)
        if client is None:
            client = _BlockingOdooClient(creds, settings.ODOO_TIMEOUT_SECONDS)
            _client_cache[creds.cache_key] = client
        return client

class OdooService:
    """Async wrapper. Every method offloads its blocking XML-RPC call to the
    anyio worker thread pool, so the event loop keeps serving other requests.

    Capacity note: anyio's default thread limiter is 40 threads per event loop,
    and each in-flight Odoo call consumes one. If you expect more than ~40
    concurrent Odoo operations, raise it during lifespan startup:

        anyio.to_thread.current_default_thread_limiter().total_tokens = 80
    """

    def __init__(self, creds: OdooCredentials) -> None:
        self._creds = creds
        self._client = _get_client(creds)

    async def _run(self, fn, *args, **kwargs):
        try:
            return await anyio.to_thread.run_sync(partial(fn, *args, **kwargs))
        except RetryError as exc:
            # tenacity gave up
            raise OdooUnavailableError(
                str(exc.last_attempt.exception())
            ) from exc

    async def test_connection(self) -> dict[str, Any]:
        uid = await self._run(self._client.authenticate, force=True)
        version = await self._run(self._client.version)
        return {
            "uid": uid,
            "server_version": version.get("server_version"),
            "ok": True,
        }

    async def get_open_purchase_orders(
        self,
        *,

```



```

        partner_ids: list[int] | None = None,
        since: dt.datetime | None = None,
        limit: int | None = None,
    ) -> list[OdooPurchaseOrder]:
        default_since = dt.datetime.now(dt.UTC) - dt.timedelta(
            days=settings.ODOO_PO_LOOKBACK_DAYS
        )
        return await self._run(
            self._client.fetch_open_purchase_orders,
            partner_ids=partner_ids,
            since=since or default_since,
            limit=limit or settings.ODOO_PO_FETCH_LIMIT,
        )

    async def search_partners(
        self, query: str | None = None, limit: int = 50
    ) -> list[OdooPartner]:
        return await self._run(self._client.search_partners, query, limit)

    async def create_vendor_bill(
        self, po_id: int, *, vendor_ref: str | None,
        invoice_date: dt.date | None, post_bill: bool = False,
    ) -> dict[str, Any]:
        return await self._run(
            self._client.create_vendor_bill_from_po, po_id,
            vendor_ref=vendor_ref, invoice_date=invoice_date, post_bill=post_bill,
        )

    async def post_note(self, po_id: int, note: str) -> bool:
        return await self._run(self._client.flag_purchase_order, po_id, note)

```

Odoo prerequisites to give the customer. The integration user needs **Purchase** → **User** and **Invoicing** → **Billing** access rights. The API key is generated under *Preferences* → *Account Security* → *New API Key*. Without the Billing right, `action_create_invoice` fails with an `AccessError` that the handler above translates into a readable message.

Mistral OCR — `app/services/ocr_service.py`

```

from __future__ import annotations

import base64
import datetime as dt
import json
import time
from decimal import Decimal, InvalidOperation
from typing import Any

import structlog
from mistralai.client import Mistral
from mistralai.client import errors as mistral_errors
from mistralai.extra import response_format_from_pydantic_model
from tenacity import (
    retry, retry_if_exception_type, stop_after_attempt, wait_exponential_jitter,
)

from app.core.config import settings
from app.core.errors import OCRError, OCRExtractionError
from app.schemas.ocr import ExtractedInvoice

logger = structlog.get_logger(__name__)

```

Import path matters. `mistralai` 2.x reorganized the package: `src/mistralai/` is now a namespace package containing only `client/` and `extra/`. The v1 form `from mistralai import Mistral` no longer resolves, and most tutorials online still show it. Use `from mistralai.client import Mistral`.

The extraction prompt is doing real work — it encodes the domain knowledge that separates a usable extraction from a plausible-looking wrong one:

```
_ANNOTATION_PROMPT = (
    "You are extracting fields from a supplier (vendor) invoice for an accounts "
    "payable system. Rules:\n"
    "1. vendor_name is the party ISSUING the invoice and expecting payment. It is "
    "usually in the letterhead. It is NOT the 'Bill To' / 'Ship To' / customer party.\n"
    "2. All dates must be ISO 8601 (YYYY-MM-DD). If the format is ambiguous "
    "(e.g. 03/04/2026) prefer the interpretation consistent with other dates on "
    "the document; otherwise assume DD/MM/YYYY for non-US addresses.\n"
    "3. Amounts must be plain decimal numbers with a dot separator, no currency "
    "symbol and no thousands separator. '1.234,56 EUR' -> 1234.56.\n"
    "4. total_amount is the final payable amount INCLUDING tax.\n"
    "5. purchase_order_reference: capture any PO number printed anywhere on the "
    "document, including headers, footers, or line item text.\n"
    "6. Return every line item in the order printed. Do not invent values; use null."
)
```

Rule 1 is the one that matters most. Left to itself the model picks whichever company name is most prominent, which on plenty of invoice templates is the *customer*. Getting this wrong sends the matching engine hunting for POs belonging to the wrong party.

```
class OcrResult:
    __slots__ = (
        "extracted", "markdown", "raw", "model", "duration_ms", "page_count",
    )

    def __init__(self, extracted, markdown, raw, model, duration_ms, page_count):
        self.extracted = extracted
        self.markdown = markdown
        self.raw = raw
        self.model = model
        self.duration_ms = duration_ms
        self.page_count = page_count

class OCRService:
    """Mistral document AI wrapper.

    Design choice: ONE call to ocr.process with document_annotation_format does
    both the OCR and the structured extraction server-side. That roughly halves
    latency and cost versus 'ocr.process -> feed the markdown to chat.parse'.
    The chat path is kept only as a fallback for when document_annotation comes
    back empty, which happens past the 8-page annotation cap.
    """

    def __init__(self, api_key: str | None = None) -> None:
        self._api_key = api_key or settings.MISTRAL_API_KEY.get_secret_value()
        self._client = Mistral(
            api_key=self._api_key, timeout_ms=settings.MISTRAL_TIMEOUT_MS
        )

    @staticmethod
    def _data_uri(content: bytes, mime_type: str) -> str:
        return f"data:{mime_type};base64,{base64.b64encode(content).decode('ascii')}}"

    @staticmethod
    def _document_payload(
        content: bytes, mime_type: str, filename: str
    ) -> dict[str, Any]:
        uri = OCRService._data_uri(content, mime_type)
        if mime_type.startswith("image/"):
            return {"type": "image_url", "image_url": uri}
        # Note: for base64 PDFs the field is still called document_url.
        return {
            "type": "document_url",
            "document_url": uri,
            "document_name": filename,
        }
```

```

@retry(
    retry=retry_if_exception_type(
        (mistral_errors.SDKError, mistral_errors.NoResponseError)
    ),
    stop=stop_after_attempt(settings.MISTRAL_MAX_RETRIES),
    wait=wait_exponential_jitter(initial=1.0, max=12.0),
    reraise=True,
)
async def _process(self, document: dict[str, Any]) -> Any:
    return await self._client.ocr.process_async(
        model=settings.MISTRAL_OCR_MODEL,
        document=document,
        # Annotation is capped at 8 pages; asking for more returns a 400.
        pages=list(range(settings.OCR_MAX_PAGES)),
        document_annotation_format=response_format_from_pydantic_model(
            ExtractedInvoice
        ),
        document_annotation_prompt=_ANNOTATION_PROMPT,
        include_image_base64=False, # we render the PDF client-side; saves MBs
        include_blocks=False,
        table_format="markdown",
    )

async def extract_invoice(
    self, *, content: bytes, mime_type: str, filename: str
) -> OcrResult:
    document = self._document_payload(content, mime_type, filename)
    started = time.perf_counter()
    try:
        response = await self._process(document)
    except mistral_errors.HTTPValidationError as exc:
        raise OCRError(f"Mistral rejected the document: {exc}") from exc
    except mistral_errors.MistralError as exc:
        raise OCRError(f"Mistral OCR call failed: {exc}") from exc
    duration_ms = int((time.perf_counter() - started) * 1000)

    markdown = "\n\n".join((p.markdown or "") for p in (response.pages or []))
    raw = response.model_dump(mode="json")

    annotation = getattr(response, "document_annotation", None)
    extracted: ExtractedInvoice | None = None
    if annotation:
        try:
            extracted = ExtractedInvoice.model_validate_json(annotation)
        except Exception:
            # The model occasionally wraps the object or appends a trailing
            # note. Try one more parse before falling back to a second call.
            try:
                extracted = ExtractedInvoice.model_validate(json.loads(annotation))
            except Exception:
                logger.warning(
                    "ocr.annotation_unparseable", sample=str(annotation)[:400]
                )

    if extracted is None:
        extracted = await self._fallback_extract(markdown)

    # Fail loudly rather than persisting a row of nulls the user cannot act on.
    if not extracted.vendor_name and not extracted.total_amount:
        raise OCRExtractionError(
            "Could not read a vendor name or total from this document. "
            "Check that it is a readable invoice."
        )

    logger.info(
        "ocr.completed",
        duration_ms=duration_ms,
        pages=len(response.pages or []),
        vendor=extracted.vendor_name,
        total=extracted.total_amount,
        lines=len(extracted.line_items),
    )
    return OcrResult(
        extracted=extracted, markdown=markdown, raw=raw,
        model=getattr(response, "model", settings.MISTRAL_OCR_MODEL),
        duration_ms=duration_ms, page_count=len(response.pages or []),
    )

```

```

async def _fallback_extract(self, markdown: str) -> ExtractedInvoice:
    """Second pass over the OCR markdown when document annotation is absent."""
    if not markdown.strip():
        raise OCRExtractionError(
            "OCR returned no text - the document may be blank or corrupt."
        )
    try:
        completion = await self._client.chat.complete_async(
            model=settings.MISTRAL_CHAT_MODEL,
            messages=[
                {"role": "system", "content": _ANNOTATION_PROMPT},
                {"role": "user", "content": f"Invoice text:\n\n{markdown[:60_000]}"},
            ],
            response_format=response_format_from_pydantic_model(ExtractedInvoice),
            temperature=0,
        )
        payload = completion.choices[0].message.content
        if isinstance(payload, list): # content chunks
            payload = "".join(getattr(c, "text", "") for c in payload)
        return ExtractedInvoice.model_validate_json(payload)
    except Exception as exc:
        raise OCRExtractionError(f"Structured extraction failed: {exc}") from exc

# ----- helpers
def to_decimal(value: Any) -> Decimal | None:
    if value is None or value == "":
        return None
    try:
        return Decimal(str(value)).quantize(Decimal("0.0001"))
    except (InvalidOperation, ValueError):
        return None

def to_date(value: str | None) -> dt.date | None:
    if not value:
        return None
    for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%m/%d/%Y", "%d-%m-%Y", "%d.%m.%Y", "%Y/%m/%d"):
        try:
            return dt.datetime.strptime(value.strip(), fmt).date()
        except ValueError:
            continue
    return None

```

Matching engine — `app/services/matching_engine.py`

A pure function of `(ExtractedInvoice, list[OdooPurchaseOrder], kb_partner_id) → MatchResult`. Zero I/O. That is what makes it unit-testable against JSON fixtures and safe to tune without a network connection.

Weights and bands

```

from __future__ import annotations

import datetime as dt
import time
from decimal import Decimal
from typing import Iterable

from rapidfuzz import fuzz

from app.core.config import settings
from app.models.match_history import MatchMethod
from app.schemas.matching import (
    ComponentScore, MatchCandidate, MatchResult, ScoreBreakdown,
)
from app.schemas.ocr import ExtractedInvoice, ExtractedLineItem
from app.schemas.odoo import OdooPOLine, OdooPurchaseOrder
from app.utils.text import (
    extract_po_references, normalize_company_name, normalize_description,
    normalize_reference,

```

```

)

# ----- weights
# Sum = 100. Components that cannot be evaluated (e.g. the invoice has no line
# items) are dropped and the remaining weights are renormalized, so a sparse
# invoice is not silently penalized down to 40/100.
W_VENDOR = 30.0
W_AMOUNT = 25.0
W_LINES = 20.0
W_REFERENCE = 15.0
W_DATE = 10.0

# Amount tolerance: relative delta -> score.
_AMOUNT_BANDS: tuple[tuple[float, float], ...] = (
    (0.001, 100.0), # <= 0.1% : rounding only
    (0.005, 97.0),  # <= 0.5%
    (0.02, 88.0),   # <= 2%   : small freight or rounding difference
    (0.05, 70.0),   # <= 5%   : plausible tax or partial delivery
    (0.10, 45.0),   # <= 10%
    (0.20, 20.0),   # <= 20%
)

# Date proximity: days between PO order date and invoice date -> score.
_DATE_BANDS: tuple[tuple[int, float], ...] = (
    (7, 100.0),
    (30, 90.0),
    (60, 75.0),
    (90, 60.0),
    (180, 35.0),
    (365, 15.0),
)

_LINE_DESC_W = 0.60
_LINE_QTY_W = 0.25
_LINE_PRICE_W = 0.15

def _f(value: Decimal | float | None) -> float:
    return float(value) if value is not None else 0.0

def _band(value: float, bands: Iterable[tuple[float, float]]) -> float:
    for threshold, score in bands:
        if value <= threshold:
            return score
    return 0.0

def _relative_delta(a: float, b: float) -> float:
    scale = max(abs(a), abs(b), 1e-9)
    return abs(a - b) / scale

```

Why weights in this order. Vendor identity is the strongest signal: if the supplier is wrong, nothing else matters. Amount is next because it is a near-unique fingerprint among a single vendor's open POs. Line items are informative but noisy — OCR mangles descriptions. Reference is weighted low *not* because it is weak (an exact PO number is decisive) but because it is usually absent; when present it saturates the component to 100 and dominates anyway.

Vendor component — where the knowledge base earns its keep

```
def score_vendor(
    invoice: ExtractedInvoice, po: OdooPurchaseOrder, kb_partner_id: int | None
) -> ComponentScore:
    # KB short-circuit: a human already taught us this alias. Nothing fuzzy can
    # beat a confirmed mapping, so it takes full marks and we stop.
    if kb_partner_id is not None and po.partner_id == kb_partner_id:
        return ComponentScore(
            name="vendor", raw=100.0, weight=W_VENDOR, weighted=W_VENDOR,
            detail=f"Knowledge-base alias -> partner #{kb_partner_id}",
        )
    if kb_partner_id is not None and po.partner_id != kb_partner_id:
        # The KB says this vendor is someone else entirely. Strong negative.
        return ComponentScore(
            name="vendor", raw=0.0, weight=W_VENDOR, weighted=0.0,
            detail=(
                f"Knowledge base maps this vendor to partner "
                f"#{kb_partner_id}, not #{po.partner_id}"
            ),
        )

    ocr_name = normalize_company_name(invoice.vendor_name)
    po_name = normalize_company_name(po.partner_name)
    if not ocr_name or not po_name:
        return ComponentScore(
            name="vendor", raw=0.0, weight=W_VENDOR, weighted=0.0,
            applicable=False, detail="No vendor name extracted.",
        )

    # token_set_ratio handles word reordering and one name being a subset of the
    # other ('ACME' vs 'ACME INTERNATIONAL'), which is the dominant real case.
    token_set = fuzz.token_set_ratio(ocr_name, po_name)
    partial = fuzz.partial_ratio(ocr_name, po_name)
    ratio = fuzz.ratio(ocr_name, po_name)
    raw = max(token_set, 0.5 * partial + 0.5 * ratio)

    # A VAT/tax id is an exact identifier - if one is present and the names are
    # already plausible, trust it over the name similarity.
    if invoice.vendor_tax_id and normalize_reference(invoice.vendor_tax_id):
        raw = max(raw, 95.0) if raw >= 60 else raw

    return ComponentScore(
        name="vendor", raw=round(raw, 2), weight=W_VENDOR,
        weighted=round(raw * W_VENDOR / 100.0, 3),
        detail=f"fuzzy '{ocr_name}' ~ '{po_name}' = {raw:.1f}",
    )
```

Amount component

```
def score_amount(
    invoice: ExtractedInvoice, po: OdooPurchaseOrder
) -> ComponentScore:
    inv_total = _f(invoice.total_amount)
    if inv_total <= 0:
        return ComponentScore(
            name="amount", raw=0.0, weight=W_AMOUNT, weighted=0.0,
            applicable=False, detail="No total extracted.",
        )

    po_total = _f(po.amount_total)
    po_untaxed = _f(po.amount_untaxed)
    inv_untaxed = _f(invoice.subtotal)

    best = _band(_relative_delta(inv_total, po_total), _AMOUNT_BANDS)
    detail = f"invoice {inv_total:.2f} vs PO total {po_total:.2f}"

    # Some vendors invoice net while the PO carries tax, or vice versa. Compare
    # every sensible pairing and keep the strongest, but discount the cross
    # comparisons so a true gross-to-gross match always outranks them.
    for label, a, b, penalty in (
        ("net~net", inv_untaxed, po_untaxed, 0.0),
        ("gross~net", inv_total, po_untaxed, 8.0),
        ("net~gross", inv_untaxed, po_total, 8.0),
    ):
        if a > 0 and b > 0:
            alt = max(_band(_relative_delta(a, b), _AMOUNT_BANDS) - penalty, 0.0)
            if alt > best:
                best, detail = alt, f"{label}: {a:.2f} vs {b:.2f}"

    # A currency mismatch means the numbers are simply not comparable.
    if (
        invoice.currency
        and po.currency_name
        and invoice.currency != po.currency_name.upper()[:3]
    ):
        best *= 0.4
        detail += f" (currency {invoice.currency} != {po.currency_name})"

    return ComponentScore(
        name="amount", raw=round(best, 2), weight=W_AMOUNT,
        weighted=round(best * W_AMOUNT / 100.0, 3), detail=detail,
    )
```

Reference component

```
def score_reference(
    invoice: ExtractedInvoice, po: OdooPurchaseOrder, ocr_text: str | None = None
) -> ComponentScore:
    po_ref = normalize_reference(po.name)
    if not po_ref:
        return ComponentScore(
            name="reference", raw=0.0, weight=W_REFERENCE, weighted=0.0,
            applicable=False, detail="PO has no name.",
        )

    # Structured extraction plus a regex sweep of the raw text, because vendors
    # print the PO number in unpredictable places.
    candidates = {normalize_reference(invoice.purchase_order_reference)}
    candidates.update(extract_po_references(ocr_text))
    candidates.discard("")

    raw, detail = 0.0, "No PO reference found on the invoice."
    if po_ref in candidates:
        raw, detail = 100.0, f"Exact PO reference {po.name} printed on invoice."
    elif any(po_ref in c or c in po_ref for c in candidates):
        raw, detail = 85.0, f"Partial PO reference match for {po.name}."
    elif candidates:
        best = max(fuzz.ratio(po_ref, c) for c in candidates)
        if best >= 80:
            raw = float(best) * 0.8
            detail = f"Fuzzy reference match {best:.0f} for {po.name}."
        else:
            # References exist but point elsewhere. Stays applicable=True: this
            # is evidence AGAINST this PO, not missing evidence.
            raw = 0.0
            detail = f"Invoice references {sorted(candidates)}, not {po.name}."

    # The vendor's own reference recorded on the PO matching our invoice number
    # is an equally strong identifier, from the other direction.
    inv_num = normalize_reference(invoice.invoice_number)
    if inv_num and po.partner_ref and normalize_reference(po.partner_ref) == inv_num:
        raw = max(raw, 100.0)
        detail += f" | PO partner_ref == invoice number {invoice.invoice_number}."

    applicable = bool(candidates) or bool(po.partner_ref)
    return ComponentScore(
        name="reference", raw=round(raw, 2), weight=W_REFERENCE,
        weighted=round(raw * W_REFERENCE / 100.0, 3),
        applicable=applicable, detail=detail,
    )
```

Line-item component

```
def _pair_score(inv_line: ExtractedLineItem, po_line: OdooPOLine) -> float:
    desc = fuzz.token_set_ratio(
        normalize_description(inv_line.description),
        normalize_description(f"{po_line.product_name or ''} {po_line.name}"),
    )

    # A vendor SKU printed on both sides is near-proof of the same item.
    if inv_line.product_code and po_line.product_code:
        if normalize_reference(inv_line.product_code) == normalize_reference(
            po_line.product_code
        ):
            desc = max(desc, 98.0)

    # 50.0 is a deliberate neutral: a missing quantity should neither reward nor
    # punish the pairing, only reduce its influence.
    qty_score = 50.0
    if inv_line.quantity is not None and po_line.product_qty:
        qty_score = _band(
            _relative_delta(inv_line.quantity, po_line.product_qty),
            ((0.001, 100.0), (0.02, 90.0), (0.10, 65.0), (0.25, 35.0)),
        )

    price_score = 50.0
    if inv_line.unit_price is not None and po_line.price_unit:
```



```

        price_score = _band(
            _relative_delta(inv_line.unit_price, float(po_line.price_unit)),
            ((0.001, 100.0), (0.01, 92.0), (0.05, 72.0), (0.15, 40.0)),
        )

    return _LINE_DESC_W * desc + _LINE_QTY_W * qty_score + _LINE_PRICE_W * price_score

def score_line_items(
    invoice: ExtractedInvoice, po: OdooPurchaseOrder
) -> ComponentScore:
    inv_lines = [li for li in invoice.line_items if (li.description or "").strip()]
    po_lines = [pl for pl in po.order_line if (pl.name or pl.product_name)]
    if not inv_lines or not po_lines:
        return ComponentScore(
            name="line_items", raw=0.0, weight=W_LINES, weighted=0.0,
            applicable=False, detail="No comparable line items.",
        )

    # Greedy global assignment: score every pair, sort descending, then take
    # pairs whose both sides are still unclaimed. O(n*m log(n*m)) with n and m
    # under ~100, so the optimal Hungarian algorithm is not worth a dependency.
    pairs = [
        (_pair_score(il, pl), i, j)
        for i, il in enumerate(inv_lines)
        for j, pl in enumerate(po_lines)
    ]
    pairs.sort(key=lambda t: t[0], reverse=True)

    used_inv: set[int] = set()
    used_po: set[int] = set()
    matched: list[float] = []
    for score, i, j in pairs:
        if score < 45.0:
            # below this it is not a match at all
            break
        if i in used_inv or j in used_po:
            continue
        used_inv.add(i)
        used_po.add(j)
        matched.append(score)

    # The denominator is max(len), not len(matched). An invoice covering 2 of a
    # PO's 10 lines scores ~20, not 100 - partial deliveries surface as a middling
    # score that prompts human review, which is exactly what should happen.
    denom = max(len(inv_lines), len(po_lines))
    raw = sum(matched) / denom if denom else 0.0
    avg = (sum(matched) / len(matched)) if matched else 0
    return ComponentScore(
        name="line_items", raw=round(raw, 2), weight=W_LINES,
        weighted=round(raw * W_LINES / 100.0, 3),
        detail=f"{len(matched)}/{denom} lines matched (avg {avg:.0f})",
    )

```

Date component

```
def score_date(
    invoice: ExtractedInvoice,
    po: OdooPurchaseOrder,
    invoice_date: dt.date | None,
) -> ComponentScore:
    if invoice_date is None or po.date_order is None:
        return ComponentScore(
            name="date", raw=0.0, weight=W_DATE, weighted=0.0,
            applicable=False, detail="Missing date on one side.",
        )
    delta_days = (invoice_date - po.date_order.date()).days
    if delta_days < 0:
        # The invoice predates the PO. Legitimate for a retro-PO, but suspicious
        # enough to push the total down and force a human look.
        raw = 40.0 if delta_days >= -14 else 5.0
        detail = f"Invoice is {abs(delta_days)}d BEFORE the PO date."
    else:
        raw = _band(float(delta_days), _DATE_BANDS)
        detail = f"Invoice {delta_days}d after PO date."
    return ComponentScore(
        name="date", raw=round(raw, 2), weight=W_DATE,
        weighted=round(raw * W_DATE / 100.0, 3), detail=detail,
    )
```

Assembly and ranking

```
def _band_label(score: float) -> str:
    if score >= settings.AUTO_CONFIRM_THRESHOLD:
        return "high"
    if score >= settings.REVIEW_THRESHOLD:
        return "medium"
    return "low"

def score_purchase_order(
    invoice: ExtractedInvoice,
    po: OdooPurchaseOrder,
    *,
    kb_partner_id: int | None,
    invoice_date: dt.date | None,
    ocr_text: str | None,
) -> ScoreBreakdown:
    components = [
        score_vendor(invoice, po, kb_partner_id),
        score_amount(invoice, po),
        score_line_items(invoice, po),
        score_reference(invoice, po, ocr_text),
        score_date(invoice, po, invoice_date),
    ]
    # Renormalize over applicable components only. Without this, an invoice with
    # no line items could never exceed 80 no matter how perfect the rest was.
    applicable = [c for c in components if c.applicable]
    applied_weight = sum(c.weight for c in applicable)
    total = (
        sum(c.weighted for c in applicable) / applied_weight * 100.0
        if applied_weight
        else 0.0
    )

    return ScoreBreakdown(
        components=components,
        total=round(min(total, 100.0), 2),
        applied_weight_total=applied_weight,
        method=MatchMethod.KB_ALIAS if kb_partner_id is not None else MatchMethod.FUZZY,
    )

def rank_candidates(
    invoice: ExtractedInvoice,
    purchase_orders: list[OdooPurchaseOrder],
    *,
```

```

kb_partner_id: int | None = None,
invoice_date: dt.date | None = None,
ocr_text: str | None = None,
top_n: int | None = None,
) -> MatchResult:
    started = time.perf_counter()
    top_n = top_n or settings.MAX_CANDIDATES_RETURNED

    pool = purchase_orders
    if kb_partner_id is not None:
        # KB hit: restrict the pool to that partner's POs. If they happen to have
        # none open, fall back to the full pool rather than returning nothing -
        # a stale alias should degrade the result, not erase it.
        narrowed = [po for po in purchase_orders if po.partner_id == kb_partner_id]
        pool = narrowed or purchase_orders

    scored: list[MatchCandidate] = []
    for po in pool:
        breakdown = score_purchase_order(
            invoice, po, kb_partner_id=kb_partner_id,
            invoice_date=invoice_date, ocr_text=ocr_text,
        )
        scored.append(
            MatchCandidate(
                purchase_order=po, score=breakdown.total,
                breakdown=breakdown, band=_band_label(breakdown.total),
            )
        )

    # Tie-break on amount so the ordering is deterministic across runs.
    scored.sort(
        key=lambda c: (c.score, _f(c.purchase_order.amount_total)), reverse=True
    )
    top = scored[:top_n]
    return MatchResult(
        candidates=top,
        best=top[0] if top and top[0].score >= 1.0 else None,
        kb_partner_id=kb_partner_id,
        kb_hit=kb_partner_id is not None,
        duration_ms=int((time.perf_counter() - started) * 1000),
    )

```

Tuning contract. The weights and bands are module constants on purpose. Build `tests/unit/test_matching_engine.py` from ~15 real invoice/PO fixture pairs and assert the **band** (`high` / `medium` / `low`), never an exact float. Asserting floats means every weight adjustment breaks the whole suite and the tests get deleted instead of maintained.

Knowledge base — `app/services/kb_service.py`

```

from __future__ import annotations

import datetime as dt
import uuid

import structlog
from rapidfuzz import fuzz, process
from sqlalchemy import select, update
from sqlalchemy.dialects.postgresql import insert as pg_insert
from sqlalchemy.ext.asyncio import AsyncSession

from app.models.vendor_knowledge_base import AliasSource, VendorKnowledgeBase
from app.utils.text import normalize_company_name

logger = structlog.get_logger(__name__)

# Deliberately tight. The KB short-circuits vendor scoring entirely, so a loose
# threshold here would confidently attach invoices to the wrong supplier.
FUZZY_KB_THRESHOLD = 92

```

```

class KnowledgeBaseService:
    def __init__(self, db: AsyncSession) -> None:
        self.db = db

    async def lookup(
        self, organization_id: uuid.UUID, raw_vendor: str | None
    ) -> VendorKnowledgeBase | None:
        """Step 1 of matching. Exact normalized hit first; a tight fuzzy pass
        second, so 'ACME INTL' still finds the 'ACME INTERNATIONAL' alias."""
        key = normalize_company_name(raw_vendor)
        if not key:
            return None

        stmt = select(VendorKnowledgeBase).where(
            VendorKnowledgeBase.organization_id == organization_id,
            VendorKnowledgeBase.normalized_key == key,
        )
        entry = (await self.db.execute(stmt)).scalar_one_or_none()
        if entry:
            logger.info(
                "kb.hit", kind="exact", key=key, partner_id=entry.odoo_partner_id
            )
            return entry

        # Fuzzy fallback. Loading the org's whole KB is fine at MVP scale
        # (hundreds of rows). Past ~10k, move this to a pg_trgm index.
        all_rows = (
            await self.db.execute(
                select(VendorKnowledgeBase).where(
                    VendorKnowledgeBase.organization_id == organization_id
                )
            )
        ).scalars().all()
        if not all_rows:
            return None

        index = {row.normalized_key: row for row in all_rows}
        hit = process.extractOne(
            key, index.keys(), scorer=fuzz.token_set_ratio,
            score_cutoff=FUZZY_KB_THRESHOLD,
        )
        if hit:
            entry = index[hit[0]]
            logger.info("kb.hit", kind="fuzzy", key=key, matched=hit[0], score=hit[1])
            return entry
        return None

    async def learn(
        self,
        *,
        organization_id: uuid.UUID,
        raw_vendor: str,
        odoo_partner_id: int,
        odoo_partner_name: str,
        odoo_partner_vat: str | None = None,
        user_id: uuid.UUID | None = None,
        source: AliasSource = AliasSource.USER_CONFIRMED,
    ) -> VendorKnowledgeBase:
        """Idempotent upsert on (organization_id, normalized_key).

        Conflict semantics: a human confirming a DIFFERENT partner for a key we
        already know overwrites it – the newest human decision wins. Doing this
        as a single ON CONFLICT statement rather than select-then-insert means
        two clerks confirming the same vendor simultaneously cannot deadlock or
        raise a duplicate-key error.
        """
        key = normalize_company_name(raw_vendor)
        if not key:
            raise ValueError("Vendor string normalizes to an empty key.")
        now = dt.datetime.now(dt.UTC)

        stmt = (
            pg_insert(VendorKnowledgeBase)
            .values(
                organization_id=organization_id,
                raw_vendor_string=raw_vendor,
            )

```

```

        normalized_key=key,
        raw_variants=[{"raw": raw_vendor, "at": now.isoformat()}],
        odoo_partner_id=odoo_partner_id,
        odoo_partner_name=odoo_partner_name,
        odoo_partner_vat=odoo_partner_vat,
        hit_count=1,
        confidence=100.0,
        source=source,
        last_used_at=now,
        created_by_user_id=user_id,
    )
    .on_conflict_do_update(
        constraint="uq_vendor_kb_org_normalized_key",
        set_= {
            "odoo_partner_id": odoo_partner_id,
            "odoo_partner_name": odoo_partner_name,
            "odoo_partner_vat": odoo_partner_vat,
            "hit_count": VendorKnowledgeBase.hit_count + 1,
            "last_used_at": now,
            "source": source,
            "updated_at": now,
        },
    )
    .returning(VendorKnowledgeBase)
)
entry = (await self.db.execute(stmt)).scalar_one()
logger.info(
    "kb.learned", key=key, partner_id=odoo_partner_id, source=source.value
)
return entry

async def touch(self, entry_id: uuid.UUID) -> None:
    """Record a use without changing the mapping – feeds the hit_count that
    the admin UI sorts by."""
    await self.db.execute(
        update(VendorKnowledgeBase)
        .where(VendorKnowledgeBase.id == entry_id)
        .values(
            hit_count=VendorKnowledgeBase.hit_count + 1,
            last_used_at=dt.datetime.now(dt.UTC),
        )
    )
)

```

Orchestrator — `app/services/invoice_service.py`

Everything above is composed here. The ordering inside `confirm()` is the part that matters.

```

class InvoiceService:
    """Coordinates: validate -> store blob -> OCR -> KB lookup -> fetch POs ->
    rank -> persist.

    Every step writes its outcome to match_history, so a failure at step N still
    leaves a row the user can see, diagnose and retry – rather than an upload
    that silently vanished.
    """

    async def upload_and_process(self, *, upload, org, user) -> MatchHistory: ...
    async def rematch(self, invoice_id, *, org) -> MatchResult: ...
    async def confirm(self, invoice_id, payload, *, org, user) -> MatchHistory: ...
    async def reject(self, invoice_id, reason, *, org, user) -> MatchHistory: ...

```

`upload_and_process` sequence

1. Sniff the MIME type from `magic bytes`, not the filename extension, and reject anything outside `ALLOWED_UPLOAD_MIME`.
2. Stream to disk while counting bytes; abort past `MAX_UPLOAD_BYTES` rather than buffering a 2 GB upload into memory first.

3. sha256 the content; a prior row with the same hash in this org is surfaced as a duplicate warning (not an error — re-uploading a corrected scan is legitimate).
4. Insert the `match_history` row as `UPLOADED`, commit. **The file is now recoverable even if everything after this fails.**
5. `PROCESSING` → `OCRService.extract_invoice` → persist `ocr_raw`, `ocr_markdown`, `extracted` and the promoted scalars. On failure: `OCR_FAILED` with `error_message`.
6. `KnowledgeBaseService.lookup(org.id, extracted.vendor_name)`.
7. `OdooService.get_open_purchase_orders`, narrowed by `partner_id` when the KB hit.
8. `rank_candidates(...)` → persist `candidates`, `score_breakdown`, `match_score`, `matched_po_id`, `match_method`.
9. `PENDING`. Return the full detail payload.

confirm sequence — order is load-bearing

1. Load the row org-scoped; assert `status in {PENDING, PUSH_FAILED, REJECTED}`.
2. Apply `payload.corrections` to the promoted columns, appending each change to `user_corrections` for the audit trail.
3. If `payload.learn_alias` and a vendor name exists → `KnowledgeBaseService.learn(...)`, set `alias_learned=True`.
4. Set `CONFIRMED`, `confirmed_by_user_id`, `confirmed_at`, `matched_po_id`, `matched_po_name`; `match_method=MANUAL` if the user overrode the top candidate.
5. `await db.commit()` — **before touching Odoo**. The learned alias and the human decision must survive an Odoo outage. This single ordering choice is what makes the system resilient rather than merely optimistic.
6. Set `PUSHING`, commit, then call `OdooService.create_vendor_bill(...)`.
7. On success → `PUSHED` plus `odoo_bill_id`, `odoo_bill_name`, `pushed_at`. On `OdooError` → `PUSH_FAILED` with `error_code` / `error_message`, increment `push_attempts`, and **return 200 with the failure recorded, not a 502**. The user's confirmation was accepted and is not lost; only the external write is retryable, via `POST /invoices/{id}/push`.